

QUESTION 16

Generate Text Embeddings and Perform Semantic Similarity Search

Technology Used: Python, Sentence Transformers, Scikit-learn

Embedding Model: all-MiniLM-L6-v2

Similarity Method: Cosine Similarity

1. Aim

To generate numerical vector representations (text embeddings) for a collection of documents and perform semantic similarity search by comparing the query embedding with document embeddings using cosine similarity.

2. Objectives

- Convert text documents into numerical vector representations.
- Convert a user query into an embedding using the same embedding model.
- Measure semantic similarity between the query and documents.
- Rank documents according to their similarity scores.
- Retrieve the most relevant document for the given query.

3. Introduction

Text embeddings are numerical representations of text that capture semantic meaning. An embedding model transforms sentences or documents into vectors in a high-dimensional space. Texts with similar meanings tend to have vectors that are closer to one another. Semantic similarity search uses these vector representations to retrieve relevant information even when the query and document do not use exactly the same words.

In this implementation, the Sentence Transformer model all-MiniLM-L6-v2 is used to generate 384-dimensional embeddings. The query embedding is compared with every document embedding using cosine similarity. The documents are then sorted from highest to lowest similarity.

4. System Architecture / Workflow

Semantic Similarity Search - Implementation Workflow

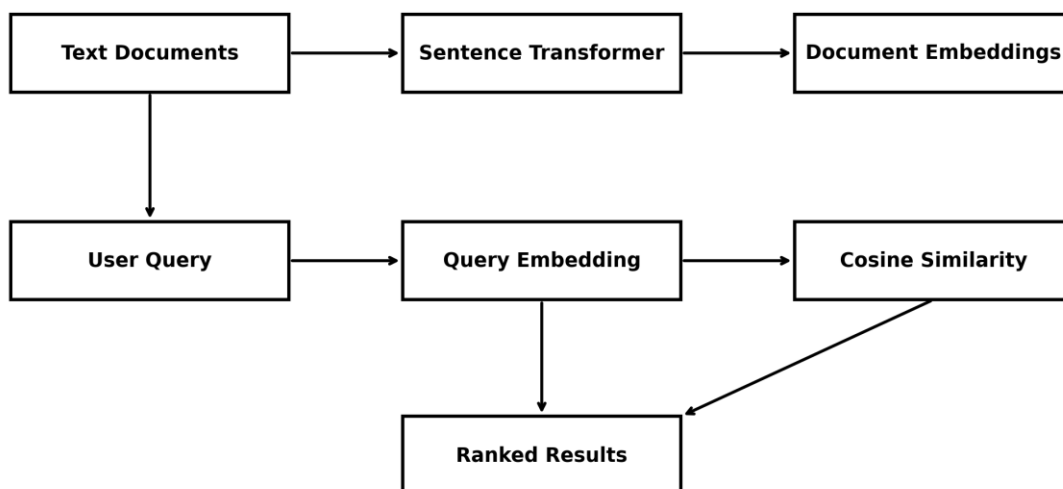


Figure 1: Semantic similarity search implementation workflow.

5. Software Requirements

Component	Specification
Programming Language	Python
IDE	VS Code / Jupyter Notebook / Google Colab
Embedding Library	Sentence Transformers
Similarity Library	Scikit-learn
Numerical Processing	NumPy (used internally by the libraries)
Embedding Model	all-MiniLM-L6-v2

6. Installation

The required libraries were installed using the following command:

```
pip install sentence-transformers scikit-learn
```

7. Algorithm / Procedure

1. Load the Sentence Transformer embedding model.
2. Define the document collection.
3. Generate embeddings for all documents.
4. Accept a natural-language query from the user.
5. Generate an embedding for the query.
6. Calculate cosine similarity between the query vector and every document vector.
7. Sort the documents in descending order of similarity.
8. Display the ranked results and the most relevant document.

8. Cosine Similarity

Cosine similarity measures the similarity between two vectors by calculating the cosine of the angle between them. A value closer to 1 indicates stronger similarity, while a value closer to 0 indicates weaker similarity.

$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \|\mathbf{B}\|)$$

9. Python Implementation

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
documents = [
    "Artificial intelligence enables machines to perform intelligent tasks.",
    "Machine learning allows computers to learn patterns from data.",
    "Deep learning uses neural networks to solve complex problems.",
    "Python is widely used for artificial intelligence and machine learning.",
    "Operating systems manage computer hardware and software resources."
]
```

```
document_embeddings = model.encode(documents)
```

```
query = input("Enter your search query: ")
query_embedding = model.encode([query])
```

```
similarity_scores = cosine_similarity(
    query_embedding,
    document_embeddings
)[0]
```

```
results = sorted(
    zip(documents, similarity_scores),
    key=lambda x: x[1],
    reverse=True
)
```

```
print("\nSEMANTIC SIMILARITY SEARCH RESULTS")
print("=" * 70)
```

```
for rank, (document, score) in enumerate(results, 1):
    print(f"\nRank {rank}")
    print(f"Similarity Score : {score:.4f}")
    print(f"Document      : {document}")
```

```
best_document, best_score = results[0]
```

```
print("\n" + "=" * 70)
print("MOST RELEVANT DOCUMENT")
print("=" * 70)
print("Document :", best_document)
print("Score   :", f"{best_score:.4f}")
print("\nSemantic search completed successfully.")
```

10. Implementation Output

```
SEMANTIC SIMILARITY SEARCH RESULTS

Rank 1
Similarity Score : 0.6891
Document       : Machine learning allows computers to learn patterns from data.

Rank 2
Similarity Score : 0.4121
Document       : Artificial intelligence enables machines to perform intelligent tasks.

Rank 3
Similarity Score : 0.3556
Document       : Operating systems manage computer hardware and software resources.

Rank 4
Similarity Score : 0.3440
Document       : Python is widely used for artificial intelligence and machine learning.

Rank 5
Similarity Score : 0.3417
Document       : Deep learning uses neural networks to solve complex problems.

MOST RELEVANT DOCUMENT
Document : Machine learning allows computers to learn patterns from data.
Score    : 0.6891

Semantic search completed successfully.
```

Figure 2: Actual implementation result for the semantic search query.

11. Experimental Result

The query used during testing was: "How do computers learn from data?" The system generated embeddings for five documents and ranked them according to cosine similarity.

Rank	Similarity Score	Document
1	0.6891	Machine learning allows computers to learn patterns from data.
2	0.4121	Artificial intelligence enables machines to perform intelligent tasks.
3	0.3556	Operating systems manage computer hardware and software resources.
4	0.3440	Python is widely used for artificial intelligence and machine learning.
5	0.3417	Deep learning uses neural networks to solve complex problems.

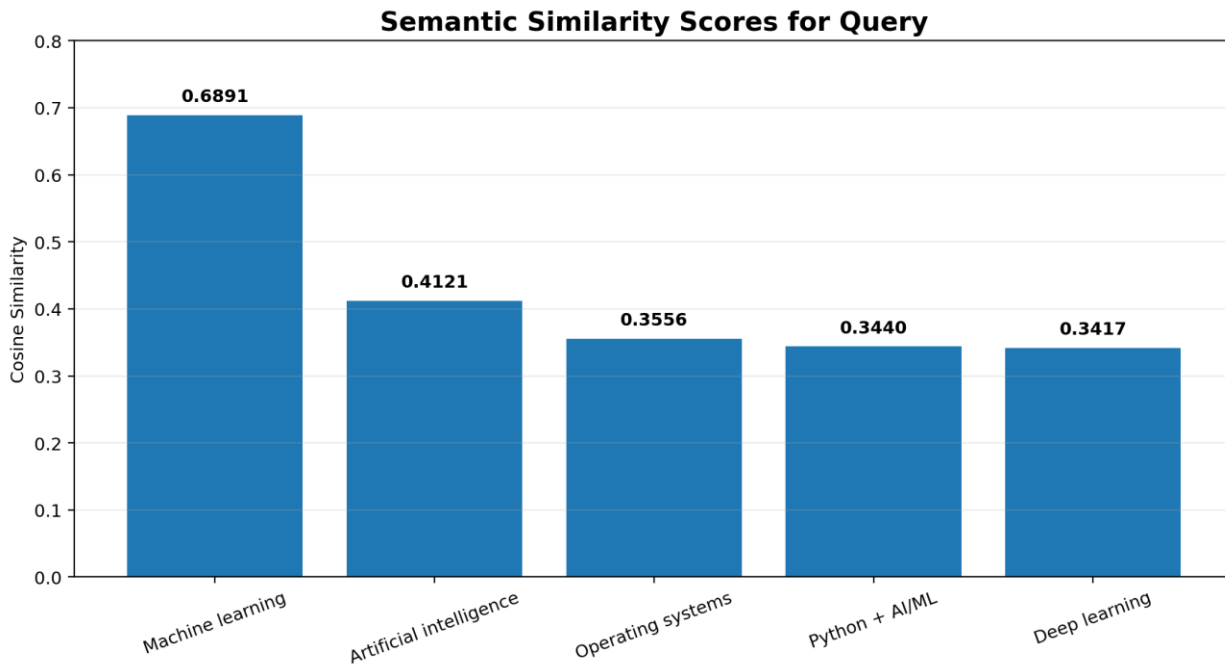


Figure 3: Similarity-score comparison of retrieved documents.

12. Result Analysis

The system successfully generated 384-dimensional embeddings for five documents. For the query “How do computers learn from data?”, the document about machine learning received the highest similarity score of 0.6891. This demonstrates that the embedding model was able to identify the document that was semantically closest to the query.

13. Advantages

- Searches according to semantic meaning rather than exact keyword matching.
- Can recognize related concepts expressed using different words.
- Produces ranked search results based on similarity.
- Provides a foundation for vector databases and RAG systems.
- Can be extended to larger document collections.

14. Applications

- Document search and information retrieval
- AI assistants and question-answering systems
- Recommendation systems
- Customer-support knowledge bases
- Retrieval-Augmented Generation (RAG)

15. Conclusion

Thus, text embeddings and semantic similarity search were successfully implemented using Sentence Transformers and cosine similarity. Documents and the user query were converted into 384-dimensional vectors, compared semantically, and ranked according to similarity. The highest scoring document was correctly retrieved, demonstrating the effectiveness of embedding-based semantic search and establishing a foundation for advanced vector retrieval and RAG applications.